(Practice) Orchestration using Airflow

💡
- Untuk akses PG Admin, run juga container dengan nama database_ny_taxi, karena container airflow ga ada pg database dan pg adminnya

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
●	database_ny_taxi	-	-	-	0.03%	20 minutes ago	■ ⋮
◑	airflow2025	-	-	-	7.76%	27 minutes ago	■ ⋮
○	video4_dockercompose	-	-	-	0%	5 months ago	▷ ⋮

▼ Setting up Airflow 2.10.4 with Docker

This is a quick, simple & less memory-intensive setup of Airflow that works on a LocalExecutor.

Only runs the webserver and the scheduler and runs the DAGs in the scheduler rather than running them in external workers:

1: Create a new sub-directory called airflow in your project dir.

Inside airflow create dags, google and logs folders.

The directory structure should look like this:

```
├── airflow
│   ├── dags
│   │
│   ├── google
│   │
│   └── logs
```

2: Create a Dockerfile.

Create a Dockerfile inside airflow folder. We are going to use this dockerfile:

```
FROM apache/airflow:2.10.4-python3.8

ENV AIRFLOW_HOME=/opt/airflow

USER airflow
```

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

WORKDIR $AIRFLOW_HOME
COPY ./google /opt/airflow/google
```

This Dockerfile creates a Docker image for Apache Airflow, installs additional Python packages from a requirements.txt file and copies google credentials into the container

---

3: Create a Docker-compose.yaml

Docker-compose.yaml should look like this:

https://github.com/ManuelGuerra1987/data-engineering-zoomcamp-notes/blob/main/2_Workflow-Orchestration-AirFlow/airflow/docker-compose.yaml

Penjelasan Docker Compose untuk Airflow

Struktur File Docker Compose

File docker-compose.yaml ini menyiapkan lingkungan Airflow dengan empat layanan utama:

1. Service Postgres

```
postgres:
    image: postgres:13
    environment:
    - POSTGRES_USER=airflow
    - POSTGRES_PASSWORD=airflow
    - POSTGRES_DB=airflow
    volumes:
        - postgres-db-volume:/var/lib/postgresql/data
    healthcheck:
        test: ["CMD", "pg_isready", "-U", "airflow"]
        interval: 5s
        retries: 5
```

- Menggunakan image PostgreSQL versi 13
- Menyiapkan database, user, dan password untuk Airflow
- Menyimpan data database di volume `postgres-db-volume`
- Menyertakan healthcheck untuk memastikan database aktif

2. Service Init-Airflow

```
init-airflow:
    build: .
    user: root
    depends_on:
    - postgres
    # ...konfigurasi lainnya...
```

- Service ini dibangun dari Dockerfile lokal (`build: .`)
- Berjalan sebagai root untuk memiliki izin yang diperlukan
- Menginisialisasi database Airflow, membuat koneksi default, dan pengguna admin
- Membuat file flag (`/opt/airflow/shared/initialized`) untuk menandakan inisialisasi selesai

3. Service Scheduler

```
scheduler:
    build: .
    # ...konfigurasi command...
    depends_on:
        - postgres
        - init-airflow
    # ...environment variables...
    volumes:
        - ./dags:/opt/airflow/dags
        - ./logs:/opt/airflow/logs
        - ./google:/opt/airflow/google:ro
        - shared-data:/opt/airflow/shared
```

- Menjalankan scheduler Airflow
- Menunggu inisialisasi selesai sebelum memulai
- Memetakan folder lokal `dags`, `logs`, dan `google` ke dalam container
- Menggunakan `LocalExecutor` (menjalankan tasks dalam proses yang sama)

4. Service Webserver

```
webserver:
    build: .
    # ...konfigurasi command...
    depends_on:
        - postgres
        - init-airflow
        - scheduler
    # ...environment variables...
    volumes:
        - ./dags:/opt/airflow/dags
        - ./logs:/opt/airflow/logs
        - ./google:/opt/airflow/google:ro
        - shared-data:/opt/airflow/shared
    user: "50000:0"
    ports:
        - "8080:8080"
    # ...healthcheck...
```

- Menjalankan antarmuka web Airflow
- Mengekspos port 8080 untuk akses web UI
- Menggunakan volume yang sama dengan scheduler

Penjelasan Section Volumes

```
volumes:
  postgres-db-volume:
  shared-data:
```

Section volumes ini **tidak kosong**, melainkan menggunakan syntax Docker Compose yang valid. Ini adalah cara untuk mendeklarasikan **named volumes** yang akan dikelola oleh Docker.

- **postgres-db-volume**: Volume untuk menyimpan data PostgreSQL
- **shared-data**: Volume untuk berbagi data antar container (terutama untuk flag inisialisasi)

Format ini (tanpa konfigurasi tambahan) adalah cara standar untuk mendefinisikan named volumes dasar di Docker Compose. Docker akan membuat dan mengelola volume-volume ini secara otomatis.

Apa yang Perlu Dilakukan?

Sebenarnya Anda **tidak perlu mengisi** section volumes ini. Definisi yang ada sudah benar dan cukup. Volume-volume tersebut akan dibuat secara otomatis saat Anda menjalankan `docker-compose up`.

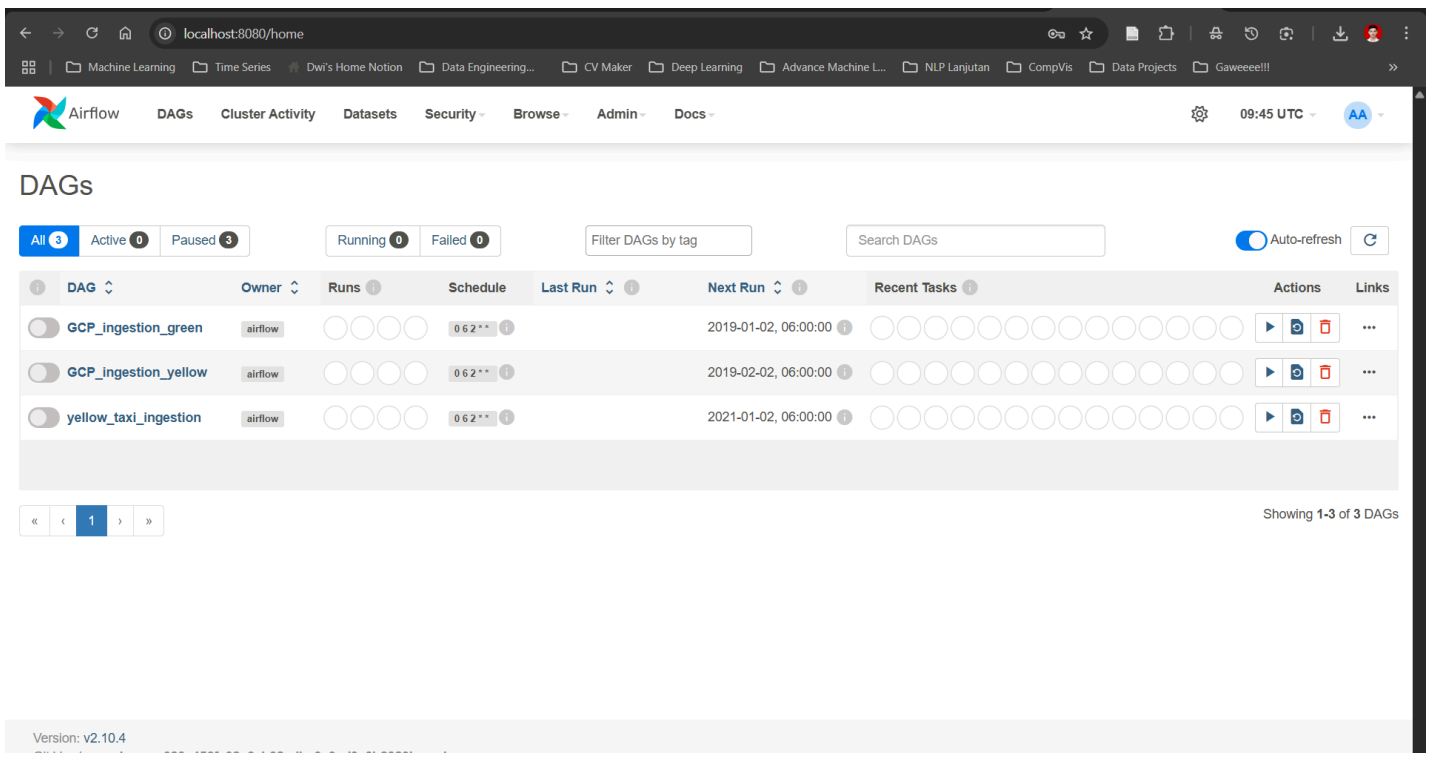
Yang perlu Anda siapkan adalah direktori lokal sesuai dengan bind mounts yang didefinisikan:

1. Buat direktori `./dags` untuk menyimpan file DAG Airflow
2. Buat direktori `./logs` untuk log Airflow (akan dibuat otomatis)
3. Buat direktori `./google` dan letakkan file credential Google Cloud (seperti yang Anda tanyakan sebelumnya)

Kemudian jalankan dengan perintah:

```
docker-compose up -d
```

Setelah container berjalan, Anda bisa mengakses web UI Airflow di `http://localhost:8080` dengan username `airflow` dan password `airflow`.



▼ Ingesting data to local Postgres

▼ Penjelasan Kode:

📋 Overview

File ini adalah implementasi DAG (Directed Acyclic Graph) Airflow untuk mengunduh, memproses, dan memasukkan data taxi kuning NYC ke dalam database PostgreSQL secara otomatis.

🔧 1. Import dan Dependencies

```
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator
import pandas as pd
from sqlalchemy import create_engine
import requests
import gzip
import shutil
```

Penjelasan Library:

- `datetime`: Untuk menangani tanggal dan waktu

- `airflow`: Framework workflow orchestration
- `pandas`: Library untuk manipulasi data dalam bentuk DataFrame
- `sqlalchemy`: ORM untuk koneksi database
- `requests`: Library untuk HTTP requests (download file)
- `gzip`: Built-in library untuk handle file terkompresi
- `shutil`: Built-in library untuk operasi file system

🗄️ 2. Konfigurasi Database

```
user = "root"
password = "root"
host = "pgdatabase"
port = "5432"
db = "ny_taxi"
```

Penjelasan: Konfigurasi koneksi PostgreSQL yang berjalan di Docker container (sesuai docker-compose-lesson1.yaml).

⬇️ 3. Fungsi Download dan Unzip

```
def download_and_unzip(csv_name_gz, csv_name, url):
    # Download the CSV.GZ file
    response = requests.get(url)
    if response.status_code == 200:
        with open(csv_name_gz, 'wb') as f_out:
            f_out.write(response.content)
    else:
        print(f"Error downloading file: {response.status_code}")
        return False

    # Unzip the CSV file
    with gzip.open(csv_name_gz, 'rb') as f_in:
        with open(csv_name, 'wb') as f_out:
            shutil.copyfileobj(f_in, f_out)

    return True
```

Detail Built-in Methods:

`requests.get(url)`:

- Mengirim HTTP GET request ke URL
- Mengembalikan Response object dengan status_code dan content

`open(filename, 'wb')`:

- Membuka file dalam mode write binary
- 'wb' = write binary mode

`response.content`:

- Mengambil content file dalam bentuk bytes

`gzip.open(filename, 'rb')`:

- Membuka file .gz dalam mode read binary
- Otomatis handle dekompresi

`shutil.copyfileobj(src, dest)`:

- Copy data dari file source ke destination
- Efisien untuk file besar karena copy chunk by chunk

🔄 4. Fungsi Process dan Insert ke Database

```
def process_and_insert_to_db(csv_name, user, password, host, port, db, table_name):
    # Connect to PostgreSQL database
    engine = create_engine(f'postgresql://{user}:{password}@{host}:{port}/{db}')
    df_iter = pd.read_csv(csv_name, iterator=True, chunksize=100000)

    # Process the first chunk
    df = next(df_iter)
    df.tpep_pickup_datetime = pd.to_datetime(df.tpep_pickup_datetime)
    df.tpep_dropoff_datetime = pd.to_datetime(df.tpep_dropoff_datetime)

    # Insert the data into the database
    df.head(n=0).to_sql(name=table_name, con=engine, if_exists='replace')
    df.to_sql(name=table_name, con=engine, if_exists='append')

    # Process the rest of the data
    while True:
        try:
            df = next(df_iter)
            df.tpep_pickup_datetime = pd.to_datetime(df.tpep_pickup_datetime)
            df.tpep_dropoff_datetime = pd.to_datetime(df.tpep_dropoff_datetime)
            df.to_sql(name=table_name, con=engine, if_exists='append')
            print('inserted another chunk')

        except StopIteration:
            print('completed')
            break
```

Detail Built-in Methods:

`create_engine(connection_string)`:

- Membuat engine koneksi database SQLAlchemy
- Format: `postgresql://user:password@host:port/database`

`pd.read_csv(filename, iterator=True, chunksize=100000)`:

- `iterator=True`: Mengembalikan iterator object, bukan DataFrame langsung
- `chunksize=100000`: Membaca file per 100,000 baris untuk menghemat memory

`next(iterator)`:

- Mengambil chunk data berikutnya dari iterator
- Throw `StopIteration` exception jika sudah selesai

`pd.to_datetime(series)`:

- Konversi string datetime ke pandas datetime object
- Penting untuk performa query database

`df.head(n=0)`:

- Mengambil 0 baris (hanya struktur/schema)
- Digunakan untuk create table structure

`df.to_sql(name, con, if_exists)`:

- `if_exists='replace'`: Drop table jika ada, lalu create baru
- `if_exists='append'`: Insert data ke table yang sudah ada

📅 5. Definisi DAG

```
dag = DAG(
    "yellow_taxi_ingestion",
    schedule_interval="0 6 2 * *",
    start_date=datetime(2021, 1, 1),
    end_date=datetime(2021, 3, 28),
    catchup=True,
    max_active_runs=1,
)
```

Parameter DAG:

- `dag_id`: "yellow_taxi_ingestion" - identifier unik
- `schedule_interval`: "0 6 2 * *" - Cron expression (jam 6 pagi tanggal 2 setiap bulan)
- `start_date`: Kapan DAG mulai aktif
- `end_date`: Kapan DAG berhenti
- `catchup=True`: Jalankan semua missed runs dari start_date
- `max_active_runs=1`: Maksimal 1 instance DAG berjalan bersamaan

🏷️ 6. Template Variables

```
table_name_template = 'yellow_taxi_{{ execution_date.strftime(\\'%Y_%m\\') }}'
csv_name_gz_template = 'output_{{ execution_date.strftime(\\'%Y_%m\\') }}.csv.gz'
csv_name_template = 'output_{{ execution_date.strftime(\\'%Y_%m\\') }}.csv'
url_template = "<https://github.com/DataTalksClub/nyc-tlc-data/releases/download/yellow/yellow_tripdata_>{{ e
xecution_date.strftime(\\'%Y-%m\\') }}.csv.gz"
```

Jinja2 Templates: Airflow menggunakan Jinja2 untuk dynamic values

- `{{ execution_date }}`: Tanggal eksekusi DAG run
- `strftime('%Y_%m')`: Format tahun_bulan (contoh: 2021_01)

⚙️ 7. Task Definition

```
# Task 1: Download
download_task = PythonOperator(
    task_id="download_and_unzip",
    python_callable=download_and_unzip,
    op_kwargs={
        'csv_name_gz': csv_name_gz_template,
        'csv_name': csv_name_template,
        'url': url_template
    },
    dag=dag
)

# Task 2: Process
process_task = PythonOperator(
```

```
    task_id="process_and_insert_to_db",
    python_callable=process_and_insert_to_db,
    op_kwargs={
        'csv_name': csv_name_template,
        'user': user,
        'password': password,
        'host': host,
        'port': port,
        'db': db,
        'table_name': table_name_template
    },
    dag=dag
)

# Task Dependencies
download_task >> process_task
```

Komponen PythonOperator:

- `task_id`: Identifier unik untuk task
- `python_callable`: Fungsi Python yang akan dieksekusi
- `op_kwargs`: Dictionary arguments yang dikirim ke fungsi
- `dag`: Reference ke DAG object

🔄 Alur Eksekusi (Workflow)

```
graph TD
    A[DAG Scheduler Trigger] --> B[Task 1: download_and_unzip]
    B --> C{Download Success?}
    C -->|Yes| D[Unzip File]
    D --> E[Task 2: process_and_insert_to_db]
    E --> F[Create DB Connection]
    F --> G[Read CSV in Chunks]
    G --> H[Process First Chunk]
    H --> I[Create Table Schema]
    I --> J[Insert First Chunk]
    J --> K[Process Remaining Chunks]
    K --> L{More Chunks?}
    L -->|Yes| M[Insert Chunk]
    M --> K
    L -->|No| N[Complete]
    C -->|No| O[Task Failed]
```

Detailed Flow:

1. **Scheduler Trigger**: DAG dijalankan sesuai schedule (tanggal 2 setiap bulan jam 6 pagi)
2. **Download Task**:
    - Download file .csv.gz dari GitHub
    - Unzip file ke format .csv
    - Return status success/failure
3. **Process Task** (hanya jalan jika download sukses):
    - Buat koneksi ke PostgreSQL
    - Baca CSV dengan chunking (100k baris per chunk)

- Proses chunk pertama untuk create table schema
- Insert data chunk by chunk sampai selesai

4. **Error Handling**:
    - Jika download gagal, task kedua tidak akan jalan
    - Exception handling untuk StopIteration saat chunking selesai

---

🎯 Key Benefits & Design Patterns

Memory Efficiency:

- Chunking mencegah out-of-memory untuk file besar
- Stream processing tanpa load seluruh file

Fault Tolerance:

- Task dependencies memastikan urutan eksekusi
- Error di satu task tidak affect task lain

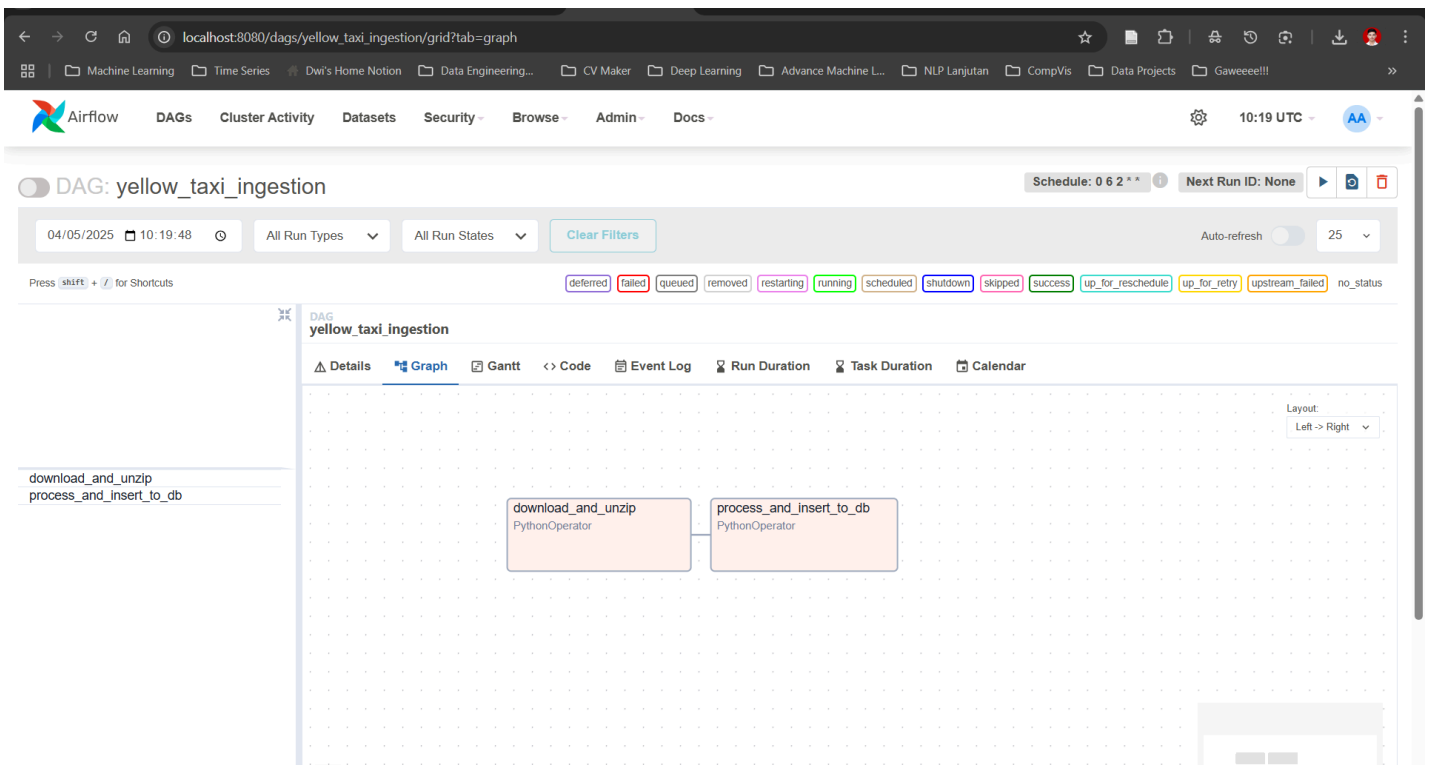
Scalability:

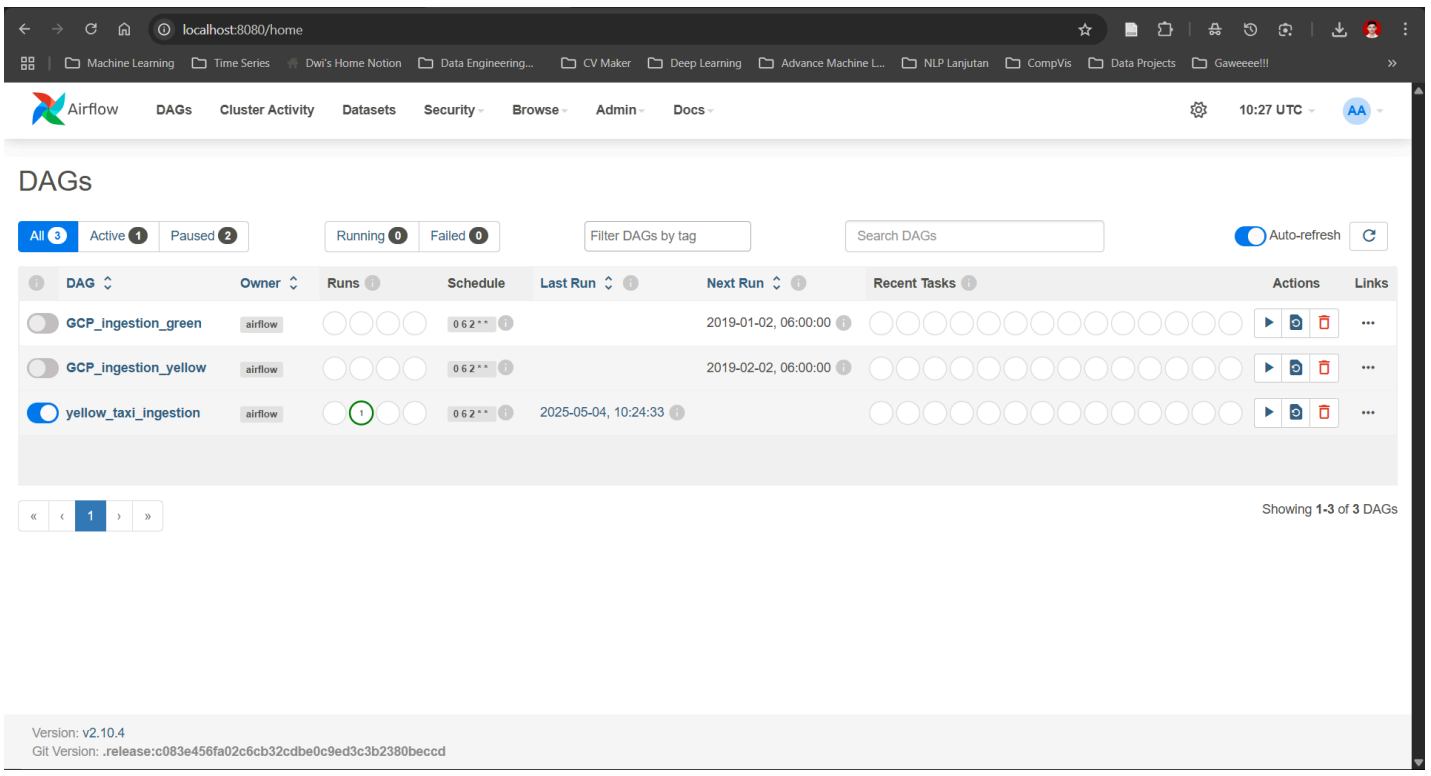
- Template-based naming untuk multiple datasets
- Parameterized functions untuk reusability

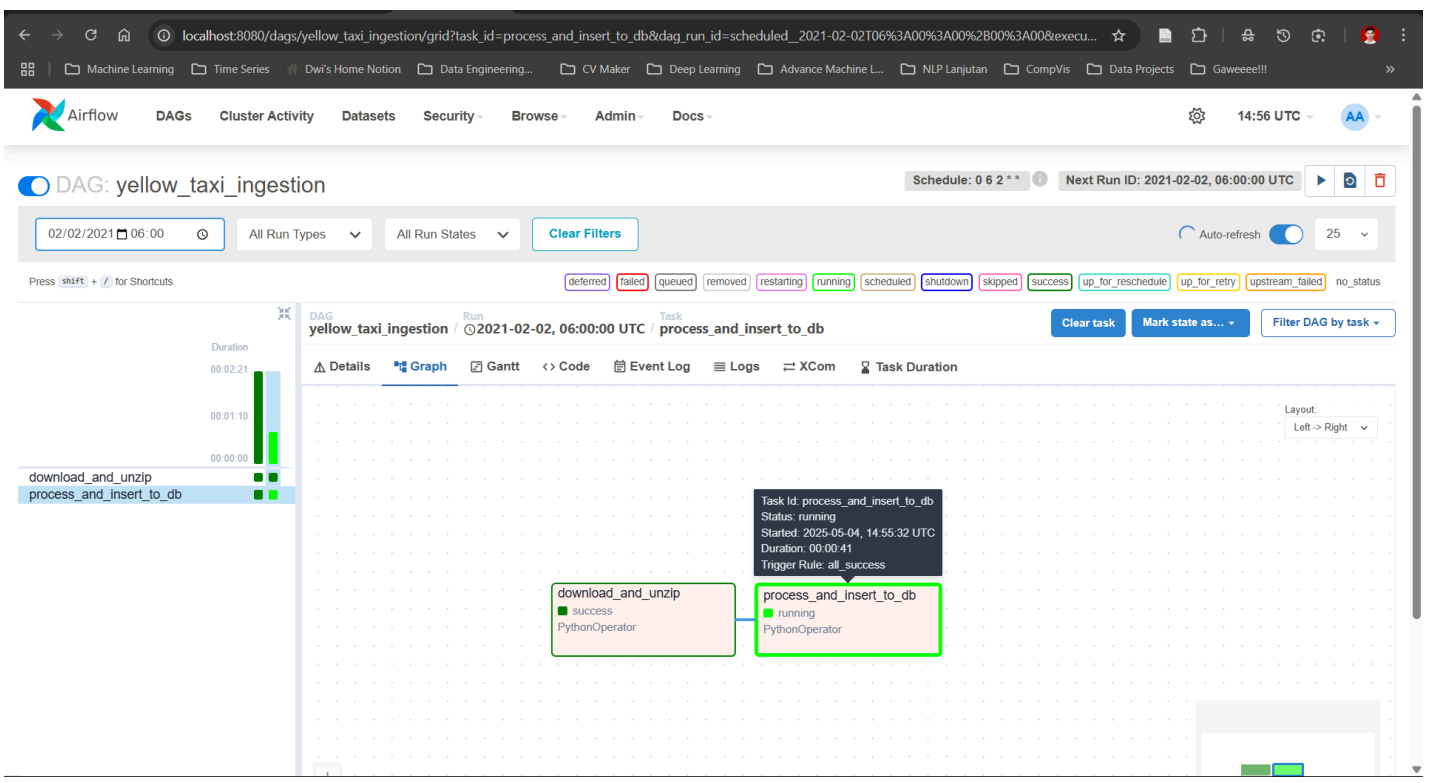
Monitoring:

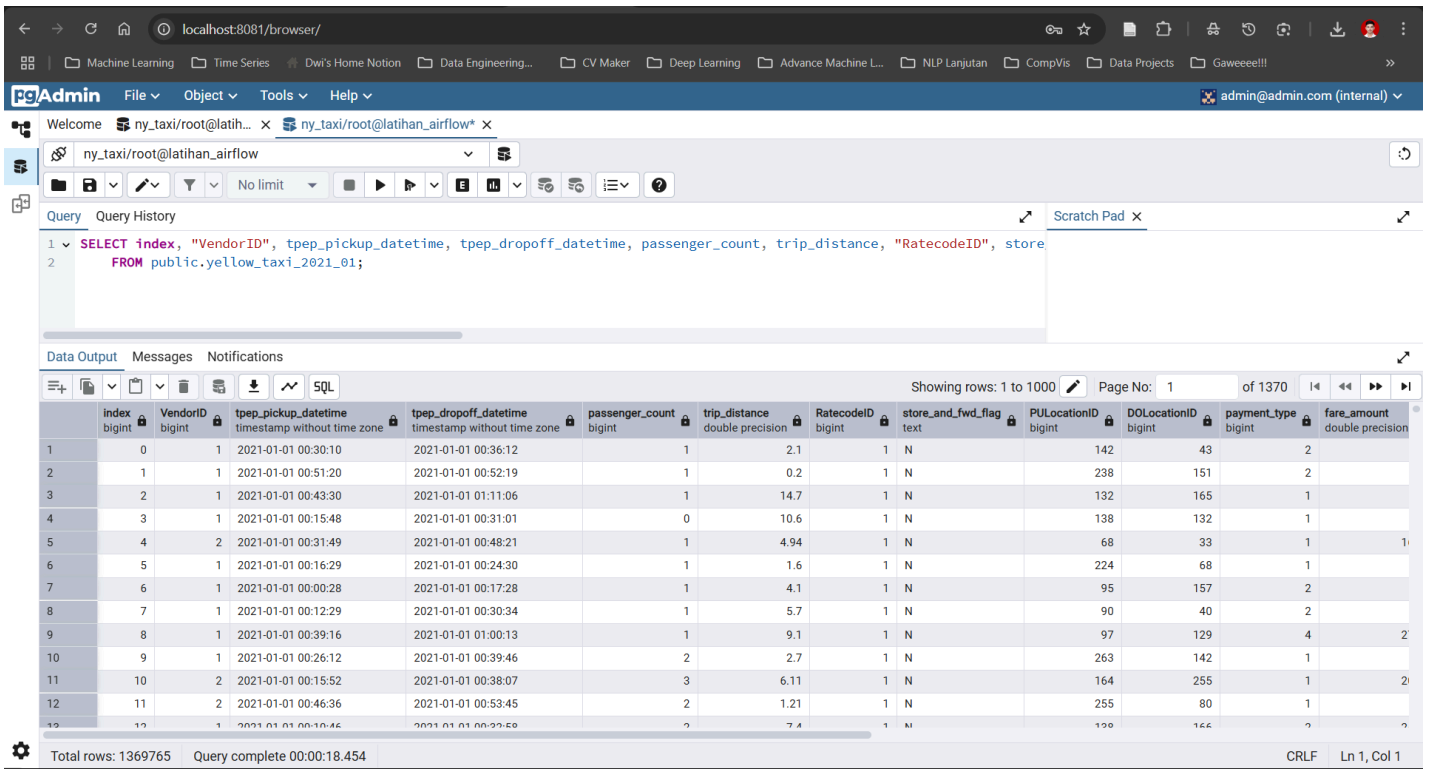
- Print statements untuk tracking progress
- Airflow UI untuk monitoring task status

▼ Dokumentasi:









▼ Ingesting data to GCP

▼ Penjelasan kode:

📋 Overview

File ini adalah implementasi DAG Airflow yang lebih advanced untuk mengunduh, memproses, dan menyimpan data taxi hijau NYC ke dalam **Google Cloud Platform (BigQuery)** dengan menggunakan **Google Cloud Storage** sebagai

staging area.

🔧 1. Import dan Dependencies

```
import os
from datetime import datetime
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.python import PythonOperator
from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator
from airflow.providers.google.cloud.hooks.gcs import GCSHook
import requests
import gzip
import shutil
import pyarrow
import pyarrow.csv
import pyarrow.parquet
```

New Libraries (Dibanding Local Version):

- `BashOperator`: Untuk eksekusi bash commands (cleanup files)
- `BigQueryInsertJobOperator`: Untuk eksekusi SQL queries di BigQuery
- `GCSHook`: Untuk interaksi dengan Google Cloud Storage
- `pyarrow`: Library untuk handling data format Parquet (columnar format)

☁️ 2. Konfigurasi GCP

```
PROJECT_ID="zoomcamp-airflow-444903"
BUCKET="zoomcamp_datalake"
BIGQUERY_DATASET = "airflow2025b"
path_to_local_home = os.environ.get("AIRFLOW_HOME", "/opt/airflow/")
```

Penjelasan:

- `PROJECT_ID`: ID project Google Cloud Platform
- `BUCKET`: Nama bucket Google Cloud Storage untuk staging
- `BIGQUERY_DATASET`: Dataset BigQuery untuk menyimpan tables
- `path_to_local_home`: Path direktori Airflow (environment variable)

📁 3. Utility Functions

A. Download Function (Sama seperti sebelumnya)

```
def download(file_gz, file_csv, url):
    response = requests.get(url)
    if response.status_code == 200:
        with open(file_gz, 'wb') as f_out:
            f_out.write(response.content)
    else:
        print(f"Error downloading file: {response.status_code}")
        return False

    with gzip.open(file_gz, 'rb') as f_in:
```

```
        with open(file_csv, 'wb') as f_out:
            shutil.copyfileobj(f_in, f_out)
```

B. Format to Parquet Function (NEW)

```
def format_to_parquet(src_file):
    table = pyarrow.csv.read_csv(src_file)

    # Handle ehail_fee column for Green taxi
    if 'ehail_fee' in table.column_names:
        table = table.set_column(
            table.schema.get_field_index('ehail_fee'),
            'ehail_fee',
            pyarrow.array(table['ehail_fee'].to_pandas().astype('float64'))
        )

    pyarrow.parquet.write_table(table, src_file.replace('.csv', '.parquet'))
```

Detail PyArrow Methods:

`pyarrow.csv.read_csv(filename)`:

- Membaca CSV file menjadi PyArrow Table (lebih efisien dari pandas)
- Table adalah struktur data columnar

`table.column_names`:

- Property yang mengembalikan list nama kolom

`table.set_column(index, name, array)`:

- Mengganti kolom di index tertentu dengan data baru
- Untuk type casting dari string ke float64

`pyarrow.parquet.write_table(table, filename)`:

- Menulis PyArrow Table ke format Parquet
- Parquet: format columnar yang compressed dan optimized untuk analytics

C. Upload to GCS Function (NEW)

```
def upload_to_gcs(bucket, object_name, local_file, gcp_conn_id="gcp-airflow"):
    hook = GCSHook(gcp_conn_id)
    hook.upload(
        bucket_name=bucket,
        object_name=object_name,
        filename=local_file,
        timeout=600
    )
```

Detail GCS Methods:

`GCSHook(gcp_conn_id)`:

- Hook untuk koneksi ke Google Cloud Storage
- `gcp_conn_id`: Reference ke connection yang sudah dikonfigurasi di Airflow

`hook.upload(bucket_name, object_name, filename, timeout)`:

- Upload file local ke GCS bucket
- `object_name`: Path/nama file di GCS

- `timeout=600`: Timeout 10 menit untuk upload

📅 4. DAG Configuration

```
dag = DAG(
    "GCP_ingestion_green",
    schedule_interval="0 6 2 * *",
    start_date=datetime(2019, 1, 1),
    end_date=datetime(2019, 12, 31),
    catchup=True,
    max_active_runs=1,
)
```

Templates yang digunakan:

```
table_name_template = 'green_taxi_{{ execution_date.strftime(\\'%Y_%m\\') }}'
file_template_csv_gz = 'output_{{ execution_date.strftime(\\'%Y_%m\\') }}.csv.gz'
file_template_csv = 'output_{{ execution_date.strftime(\\'%Y_%m\\') }}.csv'
file_template_parquet = 'output_{{ execution_date.strftime(\\'%Y_%m\\') }}.parquet'
consolidated_table_name = "green_{{ execution_date.strftime(\\'%Y\\') }}"
url_template = "<https://github.com/DataTalksClub/nyc-tlc-data/releases/download/green/green_tripdata_>{{ ex
ecution_date.strftime(\\'%Y-%m\\') }}.csv.gz"
```

⚙️ 5. Task Definitions (8 Tasks)

Task 1-3: Data Processing

```
# Task 1: Download
download_task = PythonOperator(
    task_id="download",
    python_callable=download,
    retries=10,
    dag=dag
)

# Task 2: Convert to Parquet
process_task = PythonOperator(
    task_id="format_to_parquet",
    python_callable=format_to_parquet,
    retries=10,
    dag=dag
)

# Task 3: Upload to GCS
local_to_gcs_task = PythonOperator(
    task_id="upload_to_gcs",
    python_callable=upload_to_gcs,
    retries=10,
    dag=dag
)
```

Task 4: Create Final Table

```
create_final_table_task = BigQueryInsertJobOperator(
    task_id="create_final_table",
```

```
    gcp_conn_id="gcp-airflow",
    configuration={
        "query": {
            "query": f"""
                CREATE TABLE IF NOT EXISTS `{PROJECT_ID}.{BIGQUERY_DATASET}.{consolidated_table_name}`
                (
                    unique_row_id BYTES,
                    filename STRING,
                    VendorID INT64,
                    lpep_pickup_datetime TIMESTAMP,
                    lpep_dropoff_datetime TIMESTAMP,
                    -- ... other columns
                )
            """,
            "useLegacySql": False,
        }
    },
    retries=3,
    dag=dag,
)
```

Task 5: Create External Table

```
create_external_table_task = BigQueryInsertJobOperator(
    task_id="create_external_table",
    gcp_conn_id="gcp-airflow",
    configuration={
        "query": {
            "query": f"""
                CREATE OR REPLACE EXTERNAL TABLE `{PROJECT_ID}.{BIGQUERY_DATASET}.{table_name_templat
e}_ext`
                (
                    -- Column definitions
                )
                OPTIONS (
                    uris = ['gs://{BUCKET}/raw/{file_template_parquet}'],
                    format = 'PARQUET'
                );
            """,
            "useLegacySql": False,
        }
    },
    retries=3,
    dag=dag
)
```

Task 6: Create Temp Table with Unique ID

```
create_temp_table_task = BigQueryInsertJobOperator(
    task_id="create_temp_table",
    gcp_conn_id="gcp-airflow",
    configuration={
        "query": {
            "query": f"""
                CREATE OR REPLACE TABLE `{PROJECT_ID}.{BIGQUERY_DATASET}.{table_name_template}_tmp`
                AS
                SELECT
```

```
            MD5(CONCAT(
                COALESCE(CAST(VendorID AS STRING), ""),
                COALESCE(CAST(lpep_pickup_datetime AS STRING), ""),
                COALESCE(CAST(lpep_dropoff_datetime AS STRING), ""),
                COALESCE(CAST(PULocationID AS STRING), ""),
                COALESCE(CAST(DOLocationID AS STRING), "")
            )) AS unique_row_id,
            "{file_template_parquet}" AS filename,
            *
          FROM `{PROJECT_ID}.{BIGQUERY_DATASET}.{table_name_template}_ext`;
        """,
        "useLegacySql": False,
      }
    },
    retries=3,
    dag=dag,
)
```

Task 7: Merge to Final Table

```
merge_to_final_table_task = BigQueryInsertJobOperator(
    task_id="merge_to_final_table",
    gcp_conn_id="gcp-airflow",
    configuration={
        "query": {
            "query": f"""
                MERGE INTO `{PROJECT_ID}.{BIGQUERY_DATASET}.{consolidated_table_name}` T
                USING `{PROJECT_ID}.{BIGQUERY_DATASET}.{table_name_template}_tmp` S
                ON T.unique_row_id = S.unique_row_id
                WHEN NOT MATCHED THEN
                    INSERT (unique_row_id, filename, VendorID, ...)
                    VALUES (S.unique_row_id, S.filename, S.VendorID, ...);
            """,
            "useLegacySql": False,
        }
    },
    retries=3,
    dag=dag,
)
```

Task 8: Cleanup Local Files

```
cleanup_task = BashOperator(
    task_id="cleanup_files",
    bash_command=f"rm -f {path_to_local_home}/{file_template_csv_gz} {path_to_local_home}/{file_template_cs
v} {path_to_local_home}/{file_template_parquet}",
    dag=dag,
)
```

📊 BigQuery Concepts Explained

External Table vs Native Table:

External Table:

- Data tetap di GCS, BigQuery hanya membaca schema

- Slower query performance
- Cheaper storage cost
- Good untuk exploratory analysis

Native Table:

- Data disimpan di BigQuery storage
- Faster query performance
- More expensive storage
- Optimized untuk production workloads

MERGE Statement:

- SQL DML untuk upsert operations
- Prevent duplicate data insertion
- Menggunakan `unique_row_id` sebagai deduplication key

🔄 Alur Eksekusi (Workflow)

```
graph TD
    A[Scheduler Trigger] --> B[Task 1: Download CSV.GZ]
    B --> C[Task 2: Convert to Parquet]
    C --> D[Task 3: Upload to GCS]
    D --> E[Task 4: Create Final Table Schema]
    E --> F[Task 5: Create External Table]
    F --> G[Task 6: Create Temp Table with Unique ID]
    G --> H[Task 7: Merge to Final Table]
    H --> I[Task 8: Cleanup Local Files]
```

Detailed Workflow:

1. **Download & Process**: Download compressed file, extract, convert to Parquet
2. **Upload to GCS**: Store processed file di cloud storage
3. **Create Schema**: Create final table structure di BigQuery
4. **External Table**: Create external table yang reference file di GCS
5. **Temp Processing**: Create temporary table dengan unique row ID
6. **Data Merge**: Merge new data ke final table (avoid duplicates)
7. **Cleanup**: Hapus temporary local files untuk save storage

---

🔍 Perbedaan dengan Yellow Taxi DAG

Perbedaan Utama:

Aspect	Green Taxi	Yellow Taxi
URL Source	`.../green/green_tripdata_...`	`.../yellow/yellow_tripdata_...`
Date Columns	`lpep_pickup_datetime`, `lpep_dropoff_datetime`	`tpep_pickup_datetime`, `tpep_dropoff_datetime`
Special Column	`ehail_fee` (needs type casting)	No special handling
Parquet Conversion	Full table conversion	Chunked streaming conversion
Table Prefix	`green_taxi_`, `green_`	`yellow_taxi_`, `yellow_`

Format to Parquet - Yellow Taxi (Chunked Version):

```
def format_to_parquet(src_file):
    parquet_file = src_file.replace('.csv', '.parquet')
    chunk_size=100000

    csv_reader = pyarrow.csv.open_csv(src_file, read_options=pyarrow.csv.ReadOptions(block_size=chunk_size))

    with pyarrow.parquet.ParquetWriter(parquet_file, csv_reader.schema) as writer:
        for batch in csv_reader:
            writer.write_batch(batch)
            print("another chunk inserted")
```

Yellow taxi menggunakan streaming/chunked approach untuk handle file yang lebih besar, sedangkan **Green taxi load full table** karena file lebih kecil.

🎯 Key Benefits & Advanced Patterns

Cloud-Native Architecture:

- **Data Lake Pattern**: Raw data di GCS, processed data di BigQuery
- **ELT approach**: Extract-Load-Transform di cloud
- **Cost optimization**: External tables untuk exploration, native untuk production

Data Quality & Governance:

- **Unique row ID**: Prevent duplicates dengan MD5 hash
- **Filename tracking**: Audit trail untuk source files
- **Schema evolution**: Flexible column handling

Operational Excellence:

- **Retry mechanism**: 10 retries untuk network operations
- **Resource cleanup**: Automatic file deletion
- **Connection pooling**: Reusable GCP connections

Scalability & Performance:

- **Parquet format**: Columnar storage untuk analytics
- **Partitioned processing**: Monthly table partitioning
- **Merge operations**: Efficient upsert di BigQuery

▼ Penjelasan data management:

Penjelasan Detail: Table Management & Data Flow Strategy

🗂️ Jenis-Jenis Table & Lokasi Data

1. Data Location Mapping

Location	Data Type	Purpose
Local Airflow	`.csv.gz`, `.csv`, `.parquet`	Temporary processing
GCS Bucket	`.parquet` files	Data Lake storage
BigQuery	External Table (pointer)	Query interface to GCS
BigQuery	Temp Table (native)	Processing with unique ID
BigQuery	Final Table (native)	Production data warehouse

📊 Penjelasan Setiap Table & Fungsinya

A. External Table (`_ext`) - BigQuery

```
CREATE OR REPLACE EXTERNAL TABLE `project.dataset.green_taxi_2019_01_ext`
OPTIONS (
    uris = ['gs://bucket/raw/output_2019_01.parquet'],
    format = 'PARQUET'
);
```

Karakteristik:

- ❌ **Data TIDAK disimpan di BigQuery**
- ✅ **Data tetap di GCS Bucket**
- ✅ **BigQuery hanya menyimpan schema/metadata**
- ✅ **Query langsung ke file di GCS**

Kegunaan:

- Quick access untuk exploratory analysis
- Cheaper (tidak pakai BigQuery storage)
- Slower query performance

B. Temp Table (`_tmp`) - BigQuery Native

```
CREATE OR REPLACE TABLE `project.dataset.green_taxi_2019_01_tmp`
AS
SELECT
    MD5(CONCAT(...)) AS unique_row_id,  -- Generate unique ID
    "output_2019_01.parquet" AS filename,
    *
FROM `project.dataset.green_taxi_2019_01_ext`;
```

Karakteristik:

- ✅ **Data disimpan di BigQuery storage**
- ✅ **Ada unique_row_id untuk deduplication**
- ✅ **Ada filename tracking**
- ⏱️ **Temporary - akan dihapus setelah merge**

Kegunaan:

- Processing intermediate data
- Add unique identifier
- Prepare data untuk merge

C. Final Table (`green_2019`) - BigQuery Native

```
CREATE TABLE IF NOT EXISTS `project.dataset.green_2019`
(
    unique_row_id BYTES,
    filename STRING,
    VendorID INT64,
    lpep_pickup_datetime TIMESTAMP,
    -- ... all columns
)
```

Karakteristik:

- ✅ **Data disimpan di BigQuery storage**
- ✅ **Production table untuk analytics**
- ✅ **Consolidated yearly data**
- ✅ **Optimized untuk query performance**

🔄 Alur Data Flow Lengkap

```
graph TD
    A[Internet: CSV.GZ] --> B[Local: Download CSV.GZ]
    B --> C[Local: Extract to CSV]
    C --> D[Local: Convert to Parquet]
    D --> E[GCS Bucket: Store Parquet]

    E --> F[BigQuery: External Table _ext]
    F --> G[BigQuery: Temp Table _tmp with unique_row_id]
    G --> H[BigQuery: Final Table - MERGE]

    I[Local: Cleanup Files] --> J[Complete]
    H --> I

    style E fill:#e1f5fe
    style F fill:#fff3e0
    style G fill:#fff3e0
    style H fill:#e8f5e8
```

Detailed Step-by-Step Flow:

Phase 1: Data Ingestion (Tasks 1-3)

1. **Local Processing:**

```
Internet → Local Airflow Container
├── output_2019_01.csv.gz (compressed)
├── output_2019_01.csv (extracted)
└── output_2019_01.parquet (optimized)
```

2. **Upload to Data Lake:**

```
GCS Bucket (zoomcamp_datalake)
└── raw/
    └── output_2019_01.parquet ✅ PERSISTED HERE
```

Phase 2: BigQuery Processing (Tasks 4-7)

1. **External Table Creation:**

```
BigQuery Dataset: airflow2025b
├── green_taxi_2019_01_ext (pointer to GCS)
```

   - Data masih di GCS
   - BigQuery cuma tau schema-nya

2. **Temp Table with Processing:**

```
BigQuery Dataset: airflow2025b
├── green_taxi_2019_01_ext
```

```
├── green_taxi_2019_01_tmp ✅ NEW DATA HERE
```

- Copy data dari external table ke BigQuery storage
- Add unique_row_id: MD5(VendorID + pickup_time + dropoff_time + locations)
- Add filename tracking

3. **Final Table Merge:**

```
BigQuery Dataset: airflow2025b
├── green_taxi_2019_01_ext
├── green_taxi_2019_01_tmp (temporary)
├── green_2019 ✅ PRODUCTION TABLE
```

Phase 3: Cleanup (Task 8)

1. **File Cleanup:**

```
Local Airflow: ❌ All files deleted
GCS Bucket: ✅ Parquet files remain
BigQuery: ✅ Only final table remains
```

🤔 Mengapa Butuh 3 Jenis Table?

1. External Table - Why?

Problem Solved:

- Direct access ke raw data di GCS
- Exploratory analysis tanpa cost BigQuery storage
- Bridge antara Data Lake dan Data Warehouse

Analogy:
Seperti "shortcut" di desktop yang pointing ke file di harddisk lain.

2. Temp Table - Why?

Problem Solved:

- **Deduplication:** Generate unique ID untuk prevent duplicate data
- **Data Lineage:** Track source filename
- **Performance:** Native BigQuery table query lebih cepat
- **Preprocessing:** Type casting, data cleaning

Analogy:
Seperti "staging area" di warehouse sebelum barang masuk ke stock utama.

3. Final Table - Why?

Problem Solved:

- **Historical Data:** Consolidated table untuk multiple months/years
- **Production Ready:** Optimized untuk analytics dan reporting
- **ACID Compliance:** Support untuk transactional operations
- **Partitioning & Clustering:** Better query performance

Analogy:
Seperti "inventory final" yang sudah terorganisir untuk dijual.

💰 Cost & Performance Implications

Storage Costs:

```
GCS Storage (Data Lake): $0.020/GB/month
├── Raw Parquet files stored here
├── Cheaper than BigQuery storage
└── Good for long-term archival

BigQuery Storage: $0.020/GB/month (active)
├── Temp tables (short-lived)
├── Final tables (production)
└── $0.010/GB/month (long-term inactive)

External Tables: $0 storage cost
├── No additional storage
└── Only metadata stored
```

Query Performance:

```
External Table: Slower ⚡
├── Need to read from GCS
├── No caching benefits
└── Good untuk exploratory analysis

Native Table: Faster ⚡⚡⚡
├── Data dalam BigQuery storage
├── Query optimization available
└── Production workload ready
```

🔄 MERGE Operation Explained

Mengapa Pakai MERGE?

Problem: Multiple DAG runs bisa insert duplicate data

Solution: MERGE statement = UPSERT operation

```
MERGE INTO final_table T
USING temp_table S
ON T.unique_row_id = S.unique_row_id  -- Deduplication key
WHEN NOT MATCHED THEN
  INSERT (...)  -- Only insert if unique_row_id doesn't exist
```

Benefits:

- **Idempotency:** DAG bisa dijalankan ulang tanpa duplicate data
- **Data Quality:** Ensure unique records
- **Incremental Loading:** Add new data without affecting existing records

---

📈 Monthly Processing Example

Scenario: DAG runs untuk January 2019

Before Processing:

```
GCS Bucket: empty
BigQuery Final Table (green_2019): empty
```

After January Run:

```
GCS Bucket:
└── raw/output_2019_01.parquet ✅

BigQuery:
├── green_taxi_2019_01_ext (pointer to GCS)
├── green_2019 (1 month data) ✅
```

After February Run:

```
GCS Bucket:
├── raw/output_2019_01.parquet ✅
└── raw/output_2019_02.parquet ✅

BigQuery:
├── green_taxi_2019_02_ext (pointer to GCS)
├── green_2019 (2 months data) ✅ MERGED
```

Final State:

- **Data Lake:** Individual monthly files
- **Data Warehouse:** Consolidated yearly table for analytics

---

🎯 Architecture Benefits

Lambda Architecture Pattern:

- **Data Lake (GCS):** Raw, immutable data storage
- **Data Warehouse (BigQuery):** Processed, queryable data
- **Batch Processing:** Monthly ingestion dengan Airflow orchestration

Data Governance:

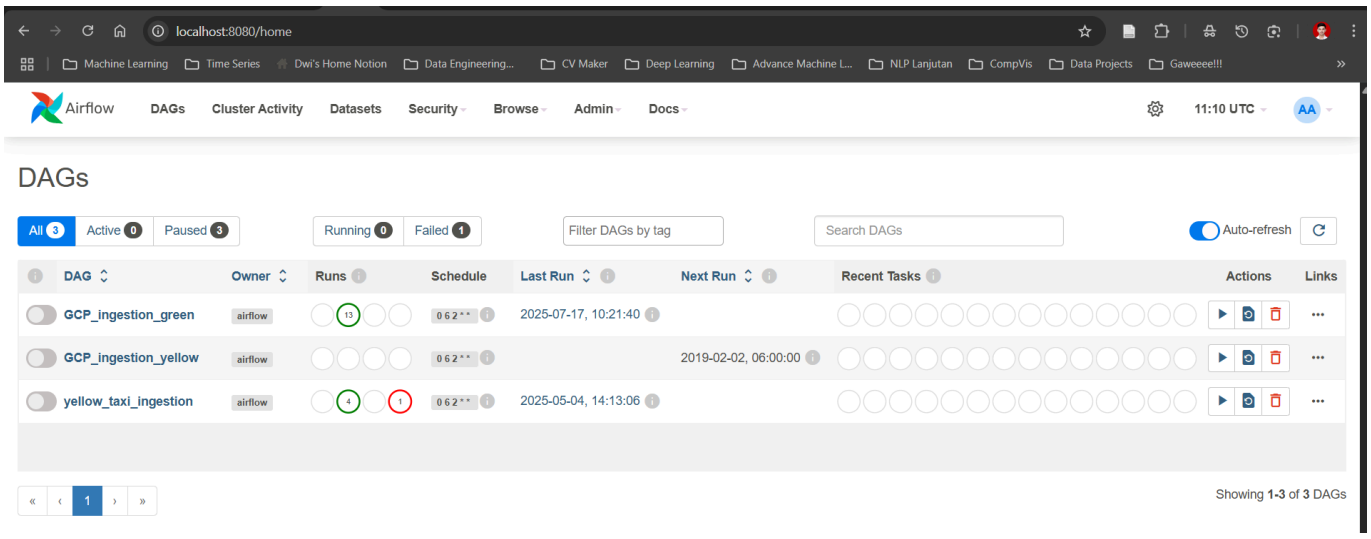
- **Source Tracking:** Filename column for audit trail
- **Data Quality:** Unique constraints dengan MD5 hash
- **Cost Optimization:** External tables untuk exploration, native untuk production

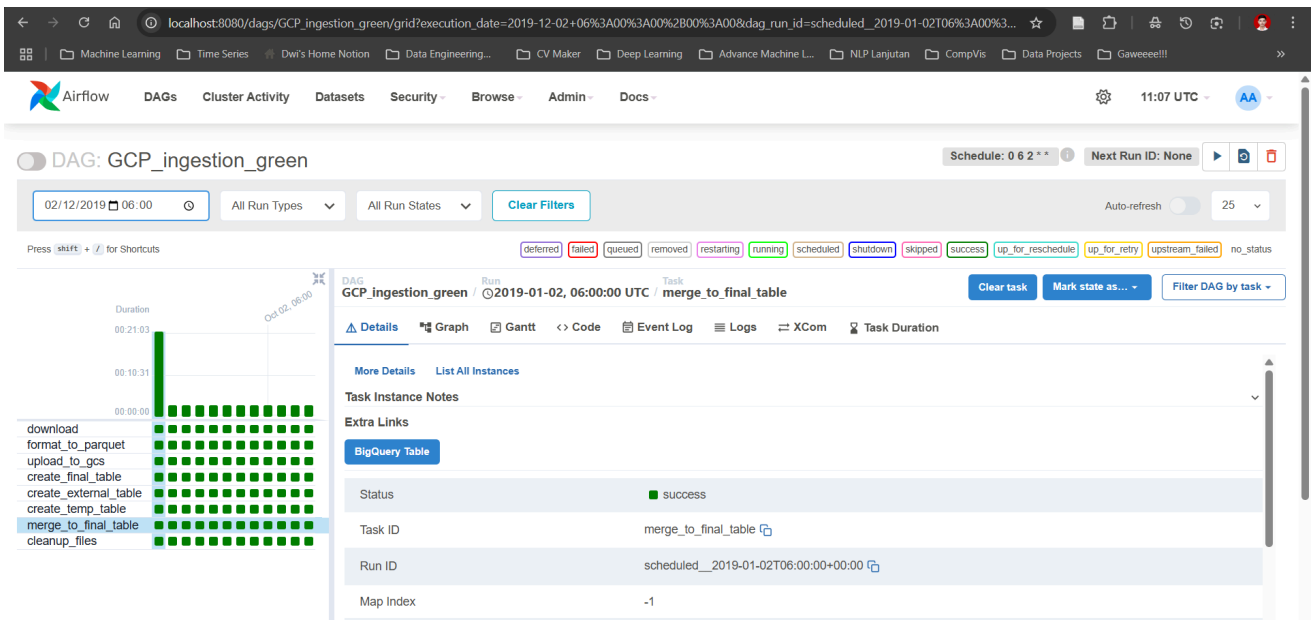
Operational Excellence:

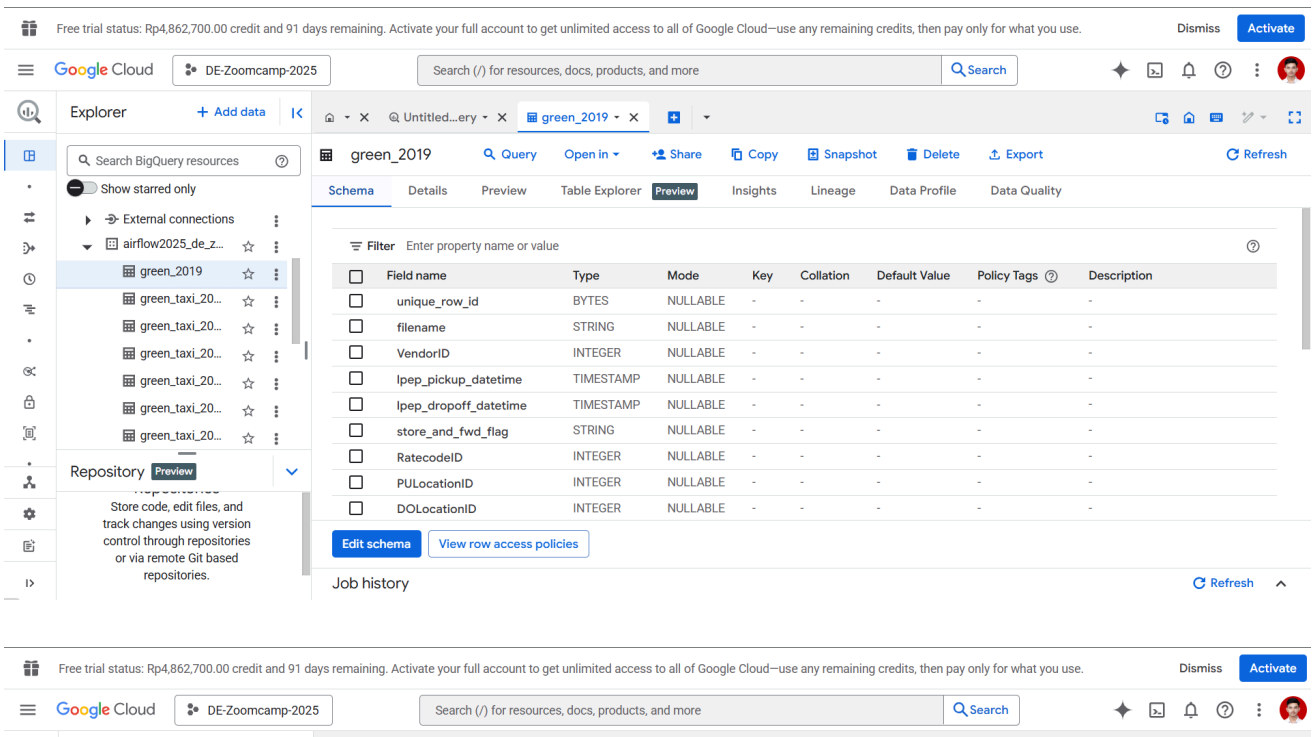
- **Fault Tolerance:** Each step independent, retryable
- **Scalability:** Handle multiple years of data
- **Monitoring:** Clear task boundaries untuk troubleshooting

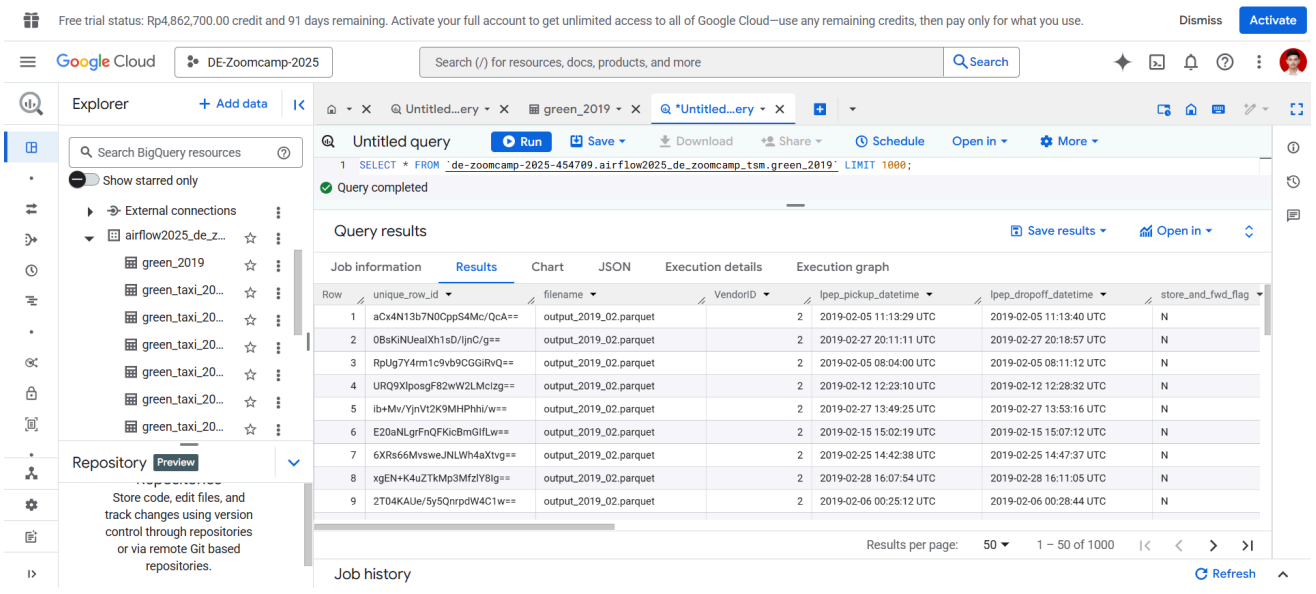
Jadi intinya: **GCS sebagai "raw storage", BigQuery sebagai "analytics engine"** dengan smart table strategy untuk optimize cost dan performance! 🚀

▼ Dokumentasi:


localhost:8080/home
Airflow
DAGs
Cluster Activity
Datasets
Security
Browse
Admin
Docs
11:10 UTC
AA
DAGs
All 3
Active 0
Paused 3
Running 0
Failed 1
Filter DAGs by tag
Search DAGs
Auto-refresh
DAG
Owner
Runs
Schedule
Last Run
Next Run
Recent Tasks
Actions
Links
GCP_ingestion_green
airflow
0 6 2 * *
2025-07-17, 10:21:40
GCP_ingestion_yellow
2019-02-02, 06:00:00
yellow_taxi_ingestion
2025-05-04, 14:13:06
Showing 1-3 of 3 DAGs


localhost:8080/dags/GCP_ingestion_green/grid?execution_date=2019-12-02+06%3A00%3A00%2B00%3A00&dag_run_id=scheduled_2019-01-02T06%3A00%3...
Airflow
DAGs
Cluster Activity
Datasets
Security
Browse
Admin
Docs
11:07 UTC
AA
DAG: GCP_ingestion_green
Schedule: 0 6 2 * *
Next Run ID: None
02/12/2019 06:00
All Run Types
All Run States
Clear Filters
Auto-refresh
25
Press shift + / for Shortcuts
deferred
failed
queued
removed
restarting
running
scheduled
shutdown
skipped
success
up_for_reschedule
up_for_retry
upstream_failed
no_status
Duration
00:21:03
00:10:31
00:00:00
Oct 02, 06:00
download
format_to_parquet
upload_to_gcs
create_final_table
create_external_table
create_temp_table
merge_to_final_table
cleanup_files
DAG
GCP_ingestion_green
Run
2019-01-02, 06:00:00 UTC
Task
merge_to_final_table
Clear task
Mark state as...
Filter DAG by task
Details
Graph
Gantt
Code
Event Log
Logs
XCom
Task Duration
More Details
List All Instances
Task Instance Notes
Extra Links
BigQuery Table
Status
success
Task ID
merge_to_final_table
Run ID
scheduled__2019-01-02T06:00:00+00:00
Map Index
-1


Free trial status: Rp4,862,700.00 credit and 91 days remaining. Activate your full account to get unlimited access to all of Google Cloud—use any remaining credits, then pay only for what you use.
Dismiss
Activate
Google Cloud
DE-Zoomcamp-2025
Search (/) for resources, docs, products, and more
Search
Explorer
Add data
Search BigQuery resources
Show starred only
External connections
airflow2025_de_z...
green_2019
green_taxi_20...
Repository
Preview
Store code, edit files, and track changes using version control through repositories or via remote Git based repositories.
Untitled...ery
green_2019
green_2019
Query
Open in
Share
Copy
Snapshot
Delete
Export
Refresh
Schema
Details
Preview
Table Explorer
Preview
Insights
Lineage
Data Profile
Data Quality
Filter
Enter property name or value
Field name
Type
Mode
Key
Collation
Default Value
Policy Tags
Description
unique_row_id
BYTES
NULLABLE
filename
STRING
NULLABLE
VendorID
INTEGER
NULLABLE
lpep_pickup_datetime
TIMESTAMP
NULLABLE
lpep_dropoff_datetime
TIMESTAMP
NULLABLE
store_and_fwd_flag
STRING
NULLABLE
RatecodeID
INTEGER
NULLABLE
PULocationID
INTEGER
NULLABLE
DOLocationID
INTEGER
NULLABLE
Edit schema
View row access policies
Job history
Refresh


Free trial status: Rp4,862,700.00 credit and 91 days remaining. Activate your full account to get unlimited access to all of Google Cloud—use any remaining credits, then pay only for what you use.
Dismiss
Activate
Google Cloud
DE-Zoomcamp-2025
Search (/) for resources, docs, products, and more
Search
Explorer
Add data
Search BigQuery resources
Show starred only
External connections
airflow2025_de_z...
green_2019
green_taxi_20...
Repository
Preview
Store code, edit files, and track changes using version control through repositories or via remote Git based repositories.
Untitled...ery
green_2019
*Untitled...ery
Untitled query
Run
Save
Download
Share
Schedule
Open in
More
1 SELECT * FROM `de-zoomcamp-2025-454709.airflow2025_de_zoomcamp_tsm.green_2019` LIMIT 1000;
Query completed
Query results
Save results
Open in
Job information
Results
Chart
JSON
Execution details
Execution graph
Row
unique_row_id
filename
VendorID
lpep_pickup_datetime
lpep_dropoff_datetime
store_and_fwd_flag
1 aCx4N13b7N0CppS4Mc/QcA== output_2019_02.parquet 2 2019-02-05 11:13:29 UTC 2019-02-05 11:13:40 UTC N
2 0BsKiNUealXh1sD/IjnC/g== output_2019_02.parquet 2 2019-02-27 20:11:11 UTC 2019-02-27 20:18:57 UTC N
3 RpUg7Y4rm1c9vb9CGGiRvQ== output_2019_02.parquet 2 2019-02-05 08:04:00 UTC 2019-02-05 08:11:12 UTC N
4 URQ9XlposgF82wW2LMcIzg== output_2019_02.parquet 2 2019-02-12 12:23:10 UTC 2019-02-12 12:28:32 UTC N
5 ib+Mv/YjnVt2K9MHPhhi/w== output_2019_02.parquet 2 2019-02-27 13:49:25 UTC 2019-02-27 13:53:16 UTC N
6 E20aNLgrFnQFKicBmGIfLw== output_2019_02.parquet 2 2019-02-15 15:02:19 UTC 2019-02-15 15:07:12 UTC N
7 6XRs66MvsweJNLWh4aXtvg== output_2019_02.parquet 2 2019-02-25 14:42:38 UTC 2019-02-25 14:47:37 UTC N
8 xgEN+K4uZTkMp3MfzlY8Ig== output_2019_02.parquet 2 2019-02-28 16:07:54 UTC 2019-02-28 16:11:05 UTC N
9 2T04KAUe/5y5QnrpdW4C1w== output_2019_02.parquet 2 2019-02-06 00:25:12 UTC 2019-02-06 00:28:44 UTC N
Results per page: 50
1 – 50 of 1000
Job history
Refresh